Suggested Teaching Guidelines for

Java Programming

PGCP-BDA, August 2026

Duration: 90 Hours: 34 Theory (T) + 36 Lab (L) + 20 Self Learning (SL)
*Note: Self-Learning comprises 4 hours of Theory and 16 hours of Lab self-learning.

Objective: To reinforce knowledge of Object Oriented Programming concepts using Core Java.

Prerequisites: Basic Programming Knowledge and OOP Concepts.

Evaluation method: Theory exam – 40%, Lab exam – 40%, Internal Assessment – 20%

List of Course Material

Text Book:

1. Java for Dummies, 7th edition, Barry Burd / Wiley

Reference book:

1. Java The Complete Reference, 9th edition, Herbert Schildt / McGraw Hill
2. Murach's Java Programming, 4th edition, Joel Murach
3. Head First Java, 3rd edition, Kathy Sierra, Bert Bates, Trisha Gee / SPD

Note: *This module adopts Java 17 (LTS) as the programming standard for all sessions, lab work, and evaluations.*

Sessions 1 & 2: Getting started with Java (4T + 2L + 2SL)

- Overview of Java
- Features of Java
- JVM Overview
- JDK and its usage
- Working with Data Types: Structure of a Java Class, Importing Packages, Difference between object reference variables and primitive variables
- Operators and Control Statements (if, if-else, for, while, switch, do-while, break and continue, ternary constructs)

Lab:

- Create a simple calculator application with 4 basic operations (+, -, *, /) using different operators and control statements (if-else/switch). Test with various input scenarios and handle edge cases like division by zero using conditional logic.

Session 3: Introduction to OOP (2T + 2L + 2SL)

- Packages and classpath
- Arrays
- Object Oriented Programming Concepts
- Object's lifecycle (creation, reassignment, garbage collection: new, finalize)

Suggested Teaching Guidelines for

Java Programming

PGCP-BDA, August 2026

Lab:

- Design a class representing a real-world entity (Employee/Student/Product) with 5-6 attributes.
- Create multiple instances, demonstrate object creation with new, reassignment of references, and observe garbage collection behavior.
- Use wrapper classes for numeric attributes.

Session 4: Pillars of OOP (2T + 4L + 2SL)

- Methods and Encapsulation: Methods, Access Modifiers, Method Overloading, Passing Data, Creating Constructors, Immutable Classes
- Class Inheritance, Abstract Classes, Inner Classes, Interface and Implementation classes.
- Understanding Polymorphism: Object vs Reference, Object Casting, Virtual Methods, Method Overriding

Lab:

- Create an inheritance hierarchy with a parent class and 2-3 child classes (e.g., Employee → Manager/Developer).
- Implement an abstract method in parent, override it in children.
- Create an inner class for additional related data.
- Demonstrate polymorphism using parent reference pointing to child objects.
- Create an anonymous inner class for a specific behavior.

Session 5: Exception Handling (2T + 2L + 2SL)

- Exception-Handling: Basics, Role of Exceptions, Types
- Using try and catch, Multiple Catch, Nested try (throw, throws, finally)
- Built-in Exceptions, Runtime Exceptions Checked Exceptions, Errors
- Creating own Exception Subclasses

Lab:

- Create a custom checked exception class (e.g., RecordNotFoundException, InvalidDataException).
- Implement search/validation methods in your entity class that throw this exception when conditions fail.
- Use try-catch-finally blocks to handle exceptions gracefully
- Demonstrate difference between checked and unchecked exceptions.
- Create a user defined exception to check whether your employee exist in your data structure and using the catch and finally block.

Suggested Teaching Guidelines for

Java Programming

PGCP-BDA, August 2026

Session 6: Strings (2T + 2L + 2SL)

- String Class and methods
- String Immutability in Java
- StringBuilder and StringBuffer class

Lab:

- Understand the difference between string literal and string object
- Understand String immutability in Java and its benefits for security and thread safety
- String interning, string constant pool memory management, and intern() method
- Build a text analyzer that counts vowels, consonants, special characters, and generates statistics
- Explore Regular expressions (regex) for pattern matching in strings

Sessions 7 & 8: Generics and Collections (4T + 4L + 4SL)

- Introduction to generics and wildcards
- Introduction to Collection framework
- List Interface and implementation classes
- Set Interface and implementation classes
- Map Interface and implementation classes

Lab:

- Create a generic repository/manager class EntityManager<T> that can work with any entity type.
- Implement CRUD operations using appropriate collection interfaces (List/Set/Map).
- Use different collection implementations (ArrayList vs LinkedList, HashSet vs TreeSet, HashMap vs TreeMap) and compare performance.
- Create generic methods for searching/filtering.

Session 9: Enums and Wrapper Class (2T + 4L)

- Enumerations
- Auto boxing
- Annotations
- java.util, java.lang package
- Wrapper classes (Boolean, Double and Integer)

Lab:

- Create an enumeration for entity status/types (e.g., EmployeeType: PERMANENT, CONTRACT, INTERN with associated salary multipliers).
- Create a custom enum for Days of Week with methods (isWeekend(), isWeekday())

- Demonstrate autoboxing/unboxing with collections (storing primitives in ArrayList). Use wrapper class methods (parseInt, valueOf, compareTo) for data conversions and comparisons.
- Store entity objects in appropriate data structures (ArrayList for dynamic list, HashMap for key-value pairs using ID as key). Use utility classes like Arrays, Collections for sorting/searching. Implement Comparable or use Comparator for custom sorting. Use Date, Random, Math classes from java.lang/util packages.

Sessions 10 & 11: Functional Programming (4T + 2L + 2SL)

- Functional Programming Overview
- Functional Interfaces
- Explore java.util.function package: Predicate, Map, Consumer, Supplier
- Lambda Expressions
- Impact of Functional programming upon Collection Framework

Lab:

- Refactor previous session's code using lambda expressions for filtering/sorting operations.

Sessions 12 & 13: Java Streams (4T + 6L + 2SL)

- Introduction to Streams
- Streams vs. Collections
- java.util.stream.Stream API
- Types of Primitive Streams: IntStream, LongStream, DoubleStream & its API
- Different operations on streams: filter, map, reduce, sort, flatMap, anyMatch, count, boxing.
- Overview of Java 8/17 Date Time API

Lab:

- Process collections using Stream API: filter entities by criteria, map to extract specific fields, reduce for aggregations (sum/average), sort by multiple attributes, use flatMap for nested collections.
- Demonstrate terminal operations (collect, forEach, count, anyMatch).
- Perform date calculations (age calculation, period between dates).

Session 14: Multithreading (2T + 2L)

- Java Concurrency: Using threads in Java, Life cycle of thread
- Thread class, thread groups
- The Runnable interface
- Synchronization

Suggested Teaching Guidelines for

Java Programming

PGCP-BDA, August 2026

Lab:

- Create a multi-threaded application simulating real-world scenario (e.g., multiple bank transactions on same account, producer-consumer problem with shared queue).
- Implement threads using both Thread class extension and Runnable interface.
- Use synchronization (synchronized methods/blocks) to prevent race conditions.

Session 15: File Handling (2T + 2L)

- The java.io Package
- Files
- Persistence of objects
- Object Serialization Methods

Lab:

- Implement file operations to save/load entity objects: serialize objects to file using ObjectOutputStream, deserialize using ObjectInputStream.
- Mark transient fields if needed.
- Create backup/restore functionality for your entity collection.
- Handle IOExceptions appropriately.

Sessions 16 & 17: Java Database Connectivity (4T + 4L + 2SL)

- Introduction of JDBC API
- JDBC Architecture
- JDBC Drivers
- Drivers
- Connection
- Statement, Prepared Statement and Result Set interfaces and their relationship to provider implementations

Lab:

- Create database tables for your entity classes with proper schema (primary keys, foreign keys, constraints).
- Implement DAO (Data Access Object) pattern with CRUD operations using JDBC: Connection establishment with appropriate driver, Statement for simple queries, PreparedStatement for parameterized queries (INSERT/UPDATE/DELETE), ResultSet processing for SELECT queries.
- Implement connection pooling concept, handle SQLExceptions, and ensure proper resource closure using try-with-resources.